LEARN

ERROR HANDLING IN BASH

Error handling helps to catch and respond to failures in a script, making it more reliable and user-friendly.

1. Exit Status

Every command returns an **exit status**:

- 0 means success
- Non-zero means an error occurred

Check the status using $?:

```
ls /nonexistentfolder
echo $?
```

2. set -e

Stops the script on any command failure:

```
set -e
```

3. Using if-statements for Error Checking

```
if cp file1.txt /backup/
then
  echo "Backup successful"
else
  echo "Backup failed"
fi
```

4. Using trap to Catch Errors

```
trap 'echo "An error occurred."; exit 1' ERR
```

5. Manually Exit with Error Code

```
if [ ! -f "important.txt" ]; then
  echo "File not found!"
  exit 1
fi
```

COMMONLY USED COMMANDS

1. echo

Used to display text or variable values

```
echo "Hello World"
name="Kanchan"
echo "My name is $name"
```

Escape sequences:

```
echo -e "Line1\nLine2"
```

2. read

Used to take input from the user

```
echo "Enter your name:"
read name
echo "Hello, $name"
```

Multiple inputs:

```
read var1 var2
```

Silent input (e.g., password):

```
read -s password
```

PRACTICE

1. Basic error check with echo:

```
mkdir mydir
if [ $? -eq 0 ]; then
  echo "Directory created"
else
  echo "Failed to create directory"
fi
```

2. Use read to take name and age:

```
echo "Enter your name:"
read name
echo "Enter your age:"
read age
echo "You are $name and you are $age years old."
```

3. Stop script on any failure:

```
set -e
echo "Step 1"
```

```
false  # this will cause the script to exit
echo "Step 2"  # this won't run
```

4. Catch error using trap:

```
trap 'echo "Oops! An error occurred."; exit 1' ERR
ls /nonexistentfolder
```

FAQ

Q1. What is $? used for?

It stores the **exit status** of the last command:

- 0: success
- non-zero: failure

Q2. How to silently read user input?

Use the -s flag:

```
read -s password
```

Q3. What does set -e do?

It causes the script to **exit immediately** if any command returns a non-zero status.

Q4. How to display special characters like newline or tab?

Use -e with echo:

```
echo -e "Line1\nLine2"
```

Q5. What does trap do?

It allows you to define commands that will be executed when certain signals are received (e.g., errors, script exit).

```
trap 'cleanup_code' EXIT
```

Q6. How to take multiple inputs using read?

```
read var1 var2
echo "You entered $var1 and $var2"
```

Q7. Can I display colored output using echo?

Yes, using ANSI codes:

```
echo -e "\e[31mThis is red text\e[0m"
```